Shared Memory Parallelism Report

November 2020

Contents

1 README **1**
1.1 parallel.c 1
1.2 sequential.c 1
1.3 Output 1

2 Implementation **2**
2.1 Array Generation and Input 2
2.2 Computation 2
2.3 Avoiding Race Conditions 2
2.4 Precision 3

3 Correctness Testing **4**

4 Scalability Investigation **5**
4.1 Efficiency 13
4.2 Speedup 14
4.3 DELETE BEFORE SUBMISSION 14

5 Further Investigation **14**

A Code **15**

1 README

1.1 parallel.c

The compiled C file (compile with the command `gcc parallel.c -lpthread -lrt`) can be run with or without the following arguments, in the stated order:

1. Dimension of input array. For example, `5` for a 5x5 array. Must be > 2 to have mutable elements Default is `18`.

2. Precision. For example, `0.1`. Default is `0.01`.

3. Number of threads to run. Any number of threads up to and including the number of mutable array elements will run, but note the maximum number of CPU cores on one Balena node (16). Default is `16`.

4. Custom array; input the name of the text file (`.txt`) containing the array as a command line argument to import the array. Array must match input dimension. Separate each value with one space only. Default is an array of random integers between 0 and the array dimension.

Commands *must* be entered in this order, and with no gaps. For example, `./parallel.out 20 0.1` is permissible for a `20x20` array and a precision of `0.1`, but `./parallel.out 20 10` will not run a `20x20` array with `10` threads.

1.2 sequential.c

The compiled C file (compile with the command `gcc sequential.c -lrt`) can be run using the same arguments as above and with the same caveats, with the exception that 'Number of Threads' may not be specified (as the program runs serially using a single thread of execution), and hence argument 4 (filename) should be entered as argument 3 if it is to be used. Identical defaults are used.

1.3 Output

By default, the provided files will output their results in the form of a labelled file: either `parallel-results-$dimension-$precision-$threadCount.txt` for `parallel.c`, or `sequential-results-$dimension-$precision.txt` for `sequential.c`. Here `$dimension` refers to the array dimension, `$precision` to the input precision, and `$threadCount` to the number of threads being used, either as defined via the command line input outlined above, or set to their default values of `18`, `0.01`, and `16` respectively.

2 Implementation

2.1 Array Generation and Input

The default array generated will be a random array of integers between 0 and the array dimension (either the input array dimension or the default array dimension of 18. To use custom arrays, see README.

The default array is generated by producing a random integer between 0 and `RAND_MAX` (a system-dependent value guaranteed to be at least 32767) and computing this value modulo the array dimension. Provided a large enough difference between `RAND_MAX` and the array dimension, this produces a pseudo-random integer between 0 and the array dimension. To ensure both the parallel and the sequential programs generate the same array, the random seed in both programs is set to the same value (in the case of the scalability investigation herein, it is set to 1), and can be altered in the code for experimental purposes.

2.2 Computation

This implementation of the relaxation technique uses `pthreads` to produce a solution, using a `do-while` loop that continues until no elements in the array change by more than the defined precision in the most recent. In each iteration, all threads operate on at least $(arraySize - 2)^2 \ /threadCount$ mutable array elements, with the 'remaining' elements (if the number of mutable elements, $arraySize - 2$, is not divisible by the number of threads) operated on by one thread each. For example, where a 20x20 array was being operated on by 16 threads, there would be 324 (18×18) mutable elements, so 12 threads would operate on 20 (324/16) array elements each and 4 (324 % 16) threads would operate on 21 each. This ensures that work is as evenly divided amongst the threads as possible, maximising the gains from parallelising operations and minimising the time threads (and hence processors) spend idle.

2.3 Avoiding Race Conditions

In order to avoid a race condition whereby threads operating on values from different iterations, two different checks were required. Firstly, two separate arrays are used to store values: one to hold the values produced from the last iteration, and one to hold those produced by threads in the current iteration. Threads then read values from the former array when calculating the mean of an element, before writing to latter; at the end of the iteration all

threads (including main) call `pthread_barrier_wait()` on the same barrier, a barrier that was initialised before the threads were created. This ensures that all threads have finished iterating their values, and the iterable array is 'finalised' for this iteration.

Each thread other than the main control thread (that runs `main()`) then calls `pthread_barrier_wait()` again - this allows for the control thread to run flag updates and to check if the program is finished before the other threads resume, while the use of barriers allows for the re-use of threads, reducing the overhead of thread creation and termination. Additionally, the two array pointers are swapped using a temporary 2D array pointer as an intermediary, providing a new 'empty' array with the same boundary values to iterate on, and an up-to-date array to read values to average from. This ensures that threads cannot 'run ahead' by multiple iterations, or operate on an array where some values have been updated and others have not.

Change the above and the below when implemented using the same thread, and when implement barriers

2.4 Precision

Iteration continues until all elements in the array, when iterated again, change by less than the given precision. This process is also parallelised. A Boolean flag, set to `true` at the beginning of each iteration. Each time a thread operates on an element a precision check is then conducted: if the absolute value of the difference between the former value and the new value of the element is greater than the precision, the flag is changed to false, and remains false for the rest of the iteration — else it remains true. The program then continues iterating until the flag is set to true even once all threads have called `pthread_barrier_wait()`, at which points the program outputs the result and exits.

3 Correctness Testing

In order to conduct correctness testing the results produced by the parallel program must be compared to a 'ground truth', in this case produced by a sequential program utilising the same algorithm. It is assumed that this sequential program produced 'correct' outputs: these outputs were manually verified for several small arrays and large precisions, and for several larger arrays was also verified against the output arrays of several other students.

In order to automate correctness testing for a large number of arrays and a variety of precisions, a bash script was produced that could schedule multiple jobs at once via a loop that iterated through various array sizes, precisions, and additionally several different thread quantities for the parallel version (in an attempt to expose any rare race conditions). Both the parallel and sequential code was then altered slightly to write the output array to a file, uniquely named for that particular run — for example, `sequential-results-1000-0.01` would identify the output of the sequential program run on an array size of 1000 and to a precision of 0.01. The content of the arrays itself was manually altered between runs of the script - for example, one set of arrays consisted of randomly allocated 1s and 0s, whilst another consisted of a set boundary value whilst the interior of the array was populated with 0.0s.

The bash `cmp` command was then used to compare the contents of the files containing the outputs of the sequential program with the parallel program for a number of different thread counts. On all tests run, on arrays from size 3–10,000, the contents of the files were identical. Hence it can be concluded that the parallel implementation provided is 'correct'.

Include code in appendix

Mention that need to change code when running correctness checker

Precision

4 Scalability Investigation

In order to assess how the parallel implementation scaled, it was necessary to develop a script that could automate running large numbers of jobs. These jobs would be performed using various array dimensions, precisions, and numbers of threads, and collect statistics including runtime, CPU utilisation, and number of iterations to complete.

Reference code in appendix

The primary array format used for these tests was a random array of values between 0 and the array dimension, as was mentioned above as the current default setting for the program, and in particular an examination of this format to a precision of 0.01. This was due to a relatively strong correlation between array dimension and the number of iterations it took to produce a solution; a random seed of 1 was selected for this reason.

A variety of array formats were experimented with, but for many of the metrics detailed in this report produced similar results, if at a different scale. For example, initial tests were conducted with an array with a boundary of 1.0, and otherwise filled with 0.0s. This array could scale to just over `7000x7000` before hitting the Balena time cap of 15 minutes, though this array also scaled in manner that was not generalisable, as iterating the array simple resulted in values that got smaller as they moved towards the centre, and so after a certain size a larger array simply resulted in a larger 'middle' section of 0.0s. In particular, the use of an array of random 1.0s and 0.0s displayed some interesting properties. Where these formats are used it has been noted.

The first (and simplest) metric to measure was simple program running time for arrays of different dimensions. Across all thread quantities measured (see Figure 1) running time increasing broadly exponentially as array size increased. This held as long as increased array size increased with the number of iterations it took to process. This is notable number of iterations increased linearly with array size (see Figure 2), though this occurred in two notable segments — between array dimensions of 0 and 250 and 250 upwards. This was linear scaling an effect of the chosen random seed, as it was not demonstrated when using alternate seeds. The rate of linear increase had a direct effect on the rate of exponential increase — as can be seen in Figure 3, an increase in array size from 0x0 to 250x250 displays a similar exponential increase, but at a far slower rate than is demonstrated from 250x250 to 550x550, when rate of linear increase in iteration number is far greater

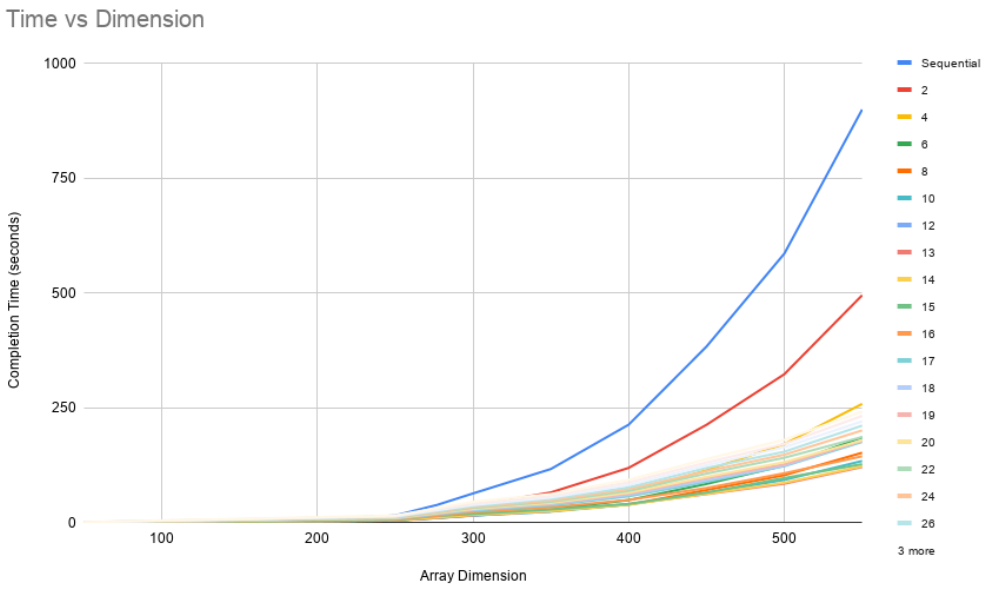

Figure 1: Program total running time vs array dimension for 1–32 threads

The compounded effects of an increase in the number of elements each thread had to process and the number of iterations taken to process the array, the former increasing quadratically with array dimension and the latter linearly, that cause this scaling. Additionally, the fact that the range of values in the array increases with array size also likely had an affect, as this exponential behaviours is not seen in other array formats, such as when arrays are formed from a random series of 1s and 0s (see Figure 4).

This exponential scaling is not demonsrated with all array formats, however

Reference data on iteration number vs array size when I talk about it above

As can be seen in Figure 2, the number of iterations it took to process an array scaled broadly linearly with array size, but with significant variation caused by the random nature of the test array. In particular, a `2300x2300` array took significantly longer to process than even significantly larger arrays, long enough that it was not possible to process a `2300x2300` array sequentially within Balena's time limit. This was the case only with the selected random seed, but further tests with another seed were not able to be conducted.

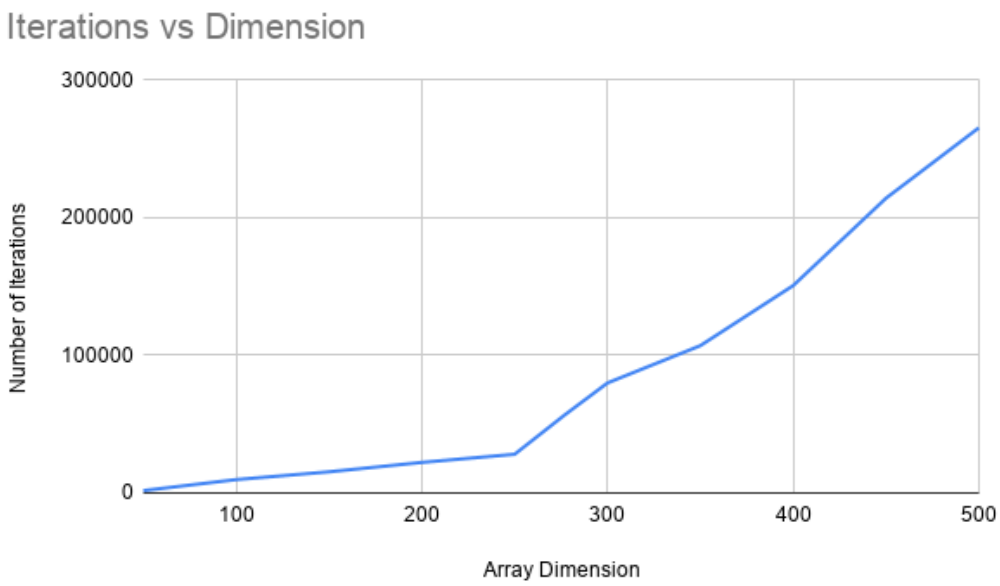

Figure 2: Number of iterations to solve array to a precision of 0.01 vs array dimension

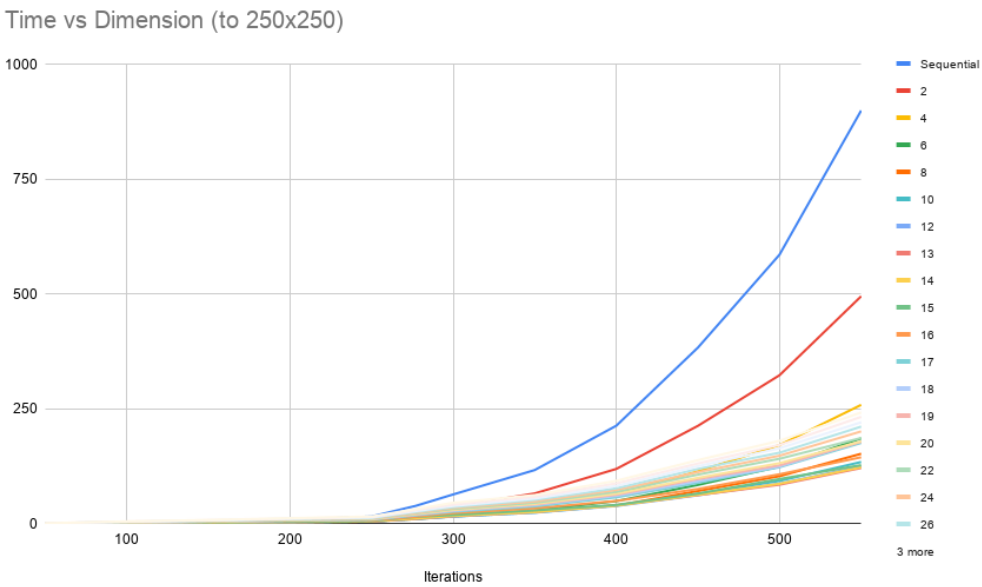

Figure 3: Program total running time vs array dimension up to a dimension of 250

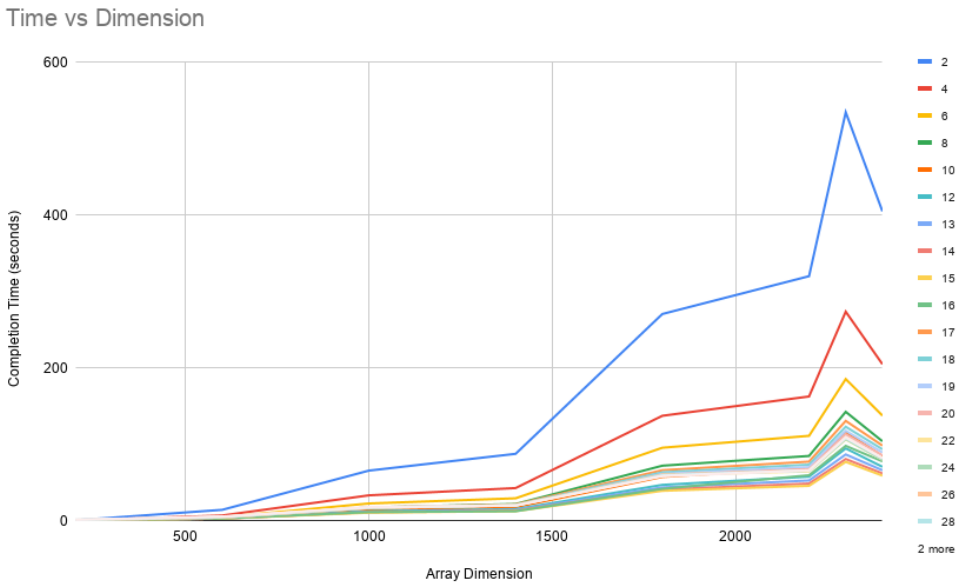

Figure 4: Program total running time vs array dimension for 1–32 threads - 1s and 0s array

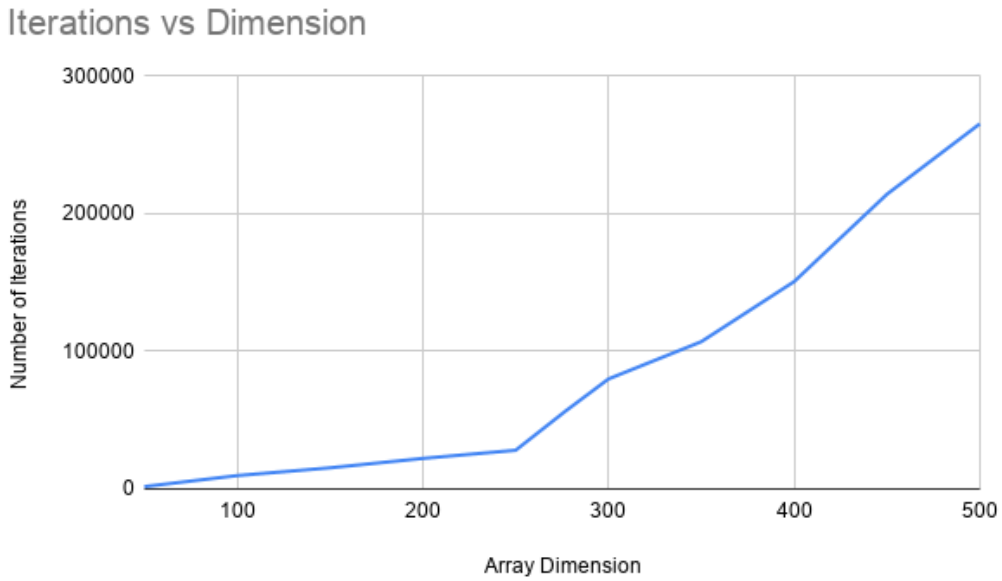

Figure 5: Number of iterations to solve array to a precision of 0.01 vs array dimension

Regarding the number of threads in use, the trend was largely rather predicable — up until a thread count of around 15, adding more threads producing faster runtimes, as can be seen in Figure 1. However, for all array dimensions measured the optimum thread count was actually 15, rather than 16, the number of processors available. This is clearly visible in 16 Threadsfig:12–16 ThreadsFigure 3, and is likely due to a increased communication between the processor cores producing an overhead less prevalent when only 15 cores where in use. An additional explanation may be control threads used by the Balena infrastructure occasionally preempting regular processing, resulting in the necessity of scheduling at a thread count of less than 17.

When the number of threads was scaled up beyond 16, the runtime dropped again, due to the overhead of scheduling multiple threads on each processor core. The slowest runtime was for 17 threads, with the runtime gradually dropping as the number of threads increased past this point, before stabilising around a point just slower than the completion time with 16 threads (see 32 Threadsfig:14–32 ThreadsFigure 4). A likely explanation for this is a relatively fixed scheduling overhead that decreases relatively as the number of threads increases.

COULD BE THAT 16 IS BEST WHEN SCALED UP EVEN MORE, MAYBE RUN SOME WITHOUT CORRESPONDING SEQUENTIAL RUNS?

One interesting, and unexpected result of the scalability investigation was the runtime per iteration. As can be seen in

reference figure here

, for 15 threads the time per iteration not only grew as array size increased, but indeed grew at a faster rate. This was the case for other thread quantities also. This was a surprising result, as it was to be expected that the time per iteration would stay largely flat, with perhaps a slight decrease as overheads such as thread management and scheduling became less significant as a proportion of runtime. The most likely explanation for this is the simple size of the arrays in memory. As the array size grew it became more difficult for the cluster to manage cache memory access as a smaller portion of the array could be stored in cache, and more time would be spent blocked as threads filled the memory bus.

Produce more data

Despite this increase in the runtime per iteration, the proportion of time or 'cycles' CPUs spent 'idle' was relatively stable across all parallel tests, displaying only a slight increase between 1 and 16 threads of just under 2% (see Figure 6 for results for 2200x2200). However, there was a marked jump in idle time between the sequential and the 2-thread parallel implementation.

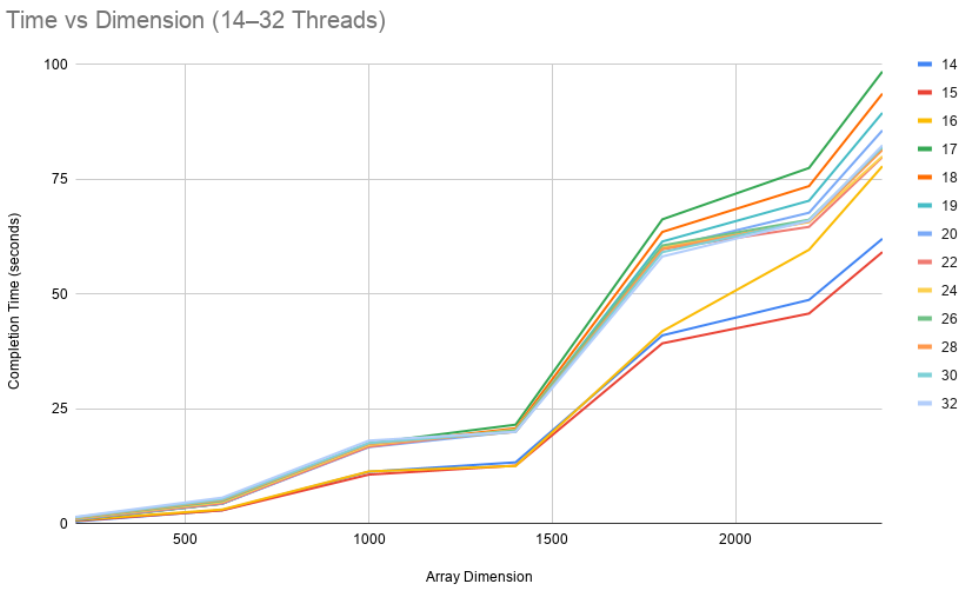

Figure 6: Program runtime vs array dimension for 14–32 threads

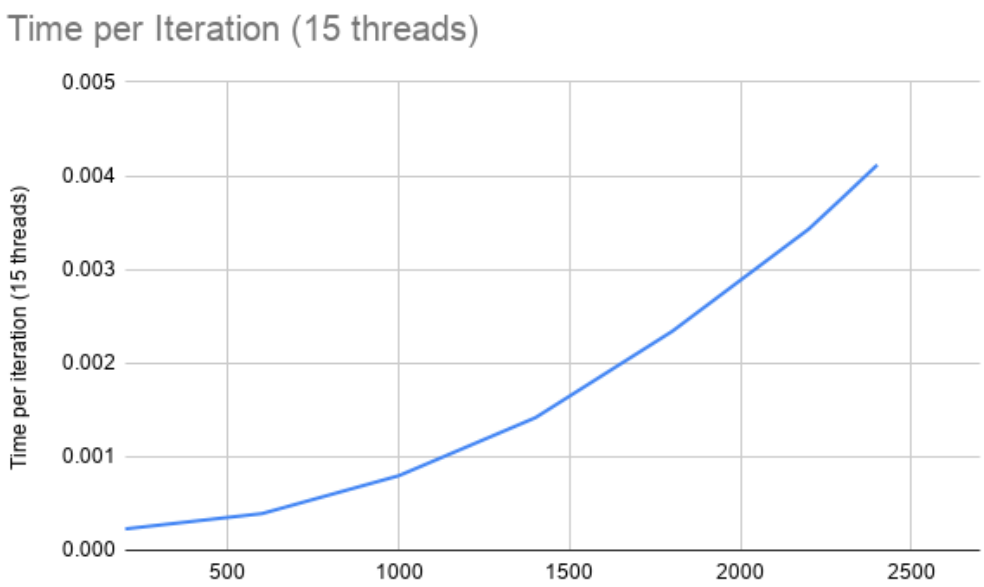

Figure 7: Time per iteration vs array dimension

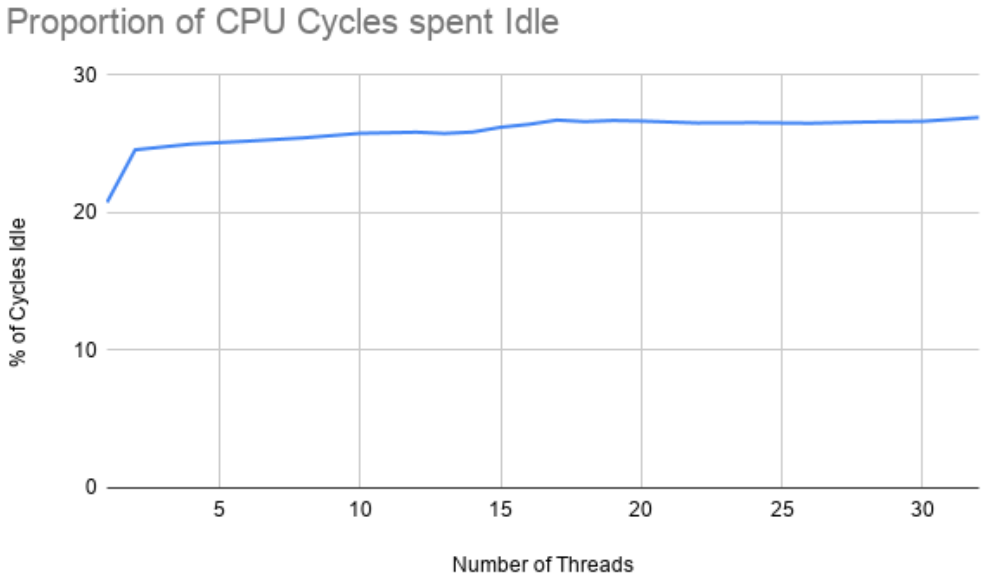

Figure 8: % of CPU Cycles spent idle (2200x2200)

This can be explained by the time spent 'waiting' for both other threads and the main control thread — a jump from 20.76% to 24.58% displayed a significant overhead in this area, an overhead that did not decrease as the number of threads was increased. Algorithmic improvements would likely be the key way to tackle this overhead.

THIS IS A LOT OF TIME - DISCUSS THIS

Despite this large amount of time spent idle, average CPU utilisation (i.e. the average number of CPUs in use) was relatively high across all runs (see Figure 7 for results for 2200x2200). Initially rising almost precisely in line with thread count, CPU utilisation was nevertheless slightly less (0.1–0.5 less) that the thread count, likely due to processor communication overheads. The highest CPU utilisation (14.219) was to be found for 15 threads, which likely explains why this thread count displayed the best performance. However, utilisation took a steep dive when more than 15 threads were in use. Average CPU utilisation was 11.057 for 16 threads, dropping to 8.491 for 17 threads and remaining relatively stable at around 10.00–10.40 from 22 threads upwards. As noted above, the effect of scheduling from 17 threads upwards likely accounts for these discrepancies.

Number of CPUs utilised

Link together scalability investigation

See TODO

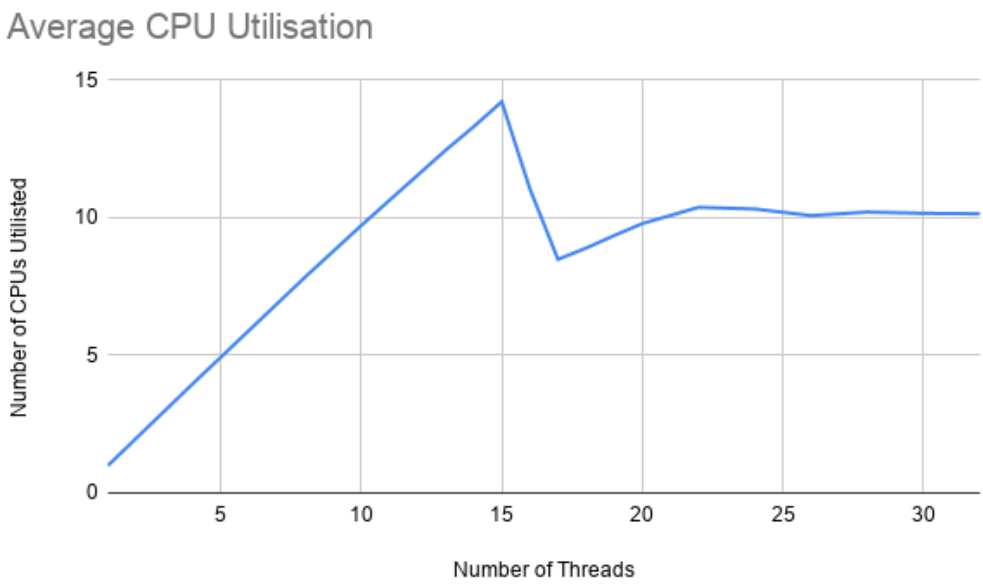

Figure 9: Average Number of CPUs Utilised (2200x2200)

Max array sizes

Used average times etc. where could

Discuss issues with 1.0s boundary array as mentioned it in the correctness testing section

4.1 Efficiency

Efficiency was a challenge to measure for this implementation, due to the constant interaction between serial and parallel elements of the program. However, an informed estimate could be provided using a wall-clock measure within the program itself. As well as measuring the total runtime of the program, the major sequential elements of the program — namely that serial processing prior to thread creation, as well as those periods when threads were blocked by wait functions — could be timed, and hence the total parallel runtime be deduced by the difference between the two.

Efficiency on the largest array size measured (2400x2400) was

Gustafson

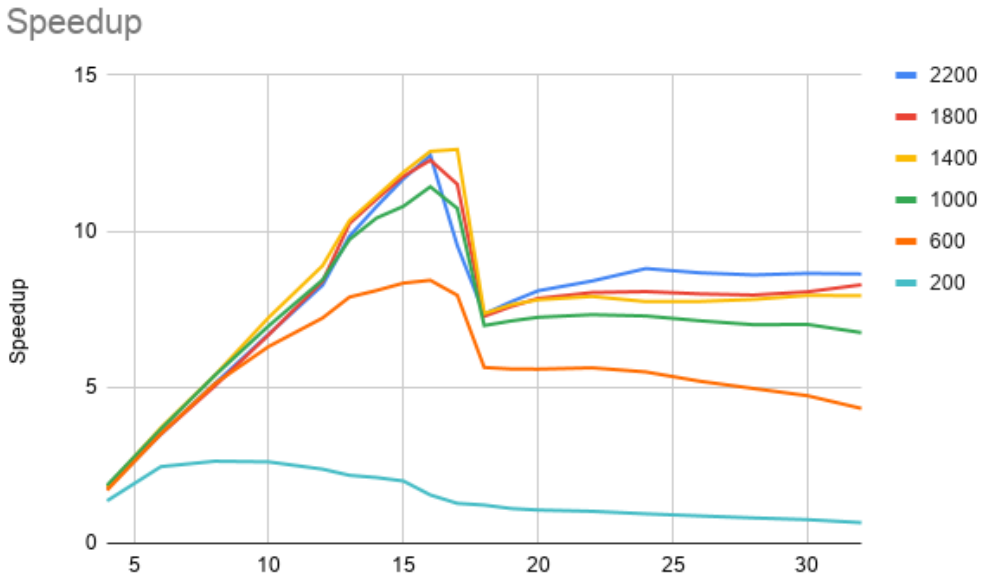

Figure 10: Speedup vs Thread Count for a various array dimensions

4.2 Speedup

Speedup less than hoped for as many of the improvements discovered for parallel could also be implemented in sequential, so despite faster speed

Speedup increases alongside thread count up to a maximum at 15 threads — the maximum speedup measured was 12.44 for an 2200x2200 array and 15 threads. The increase in speedup with thread count was expected, as overheads such as thread creation and serial elements of the program became relatively smaller in comparison to time spent processing in parallel. A speedup of 12.44 was also much greater than anticipated — relatively close to Amdahl's limit

How speedup changes with array size

4.3 DELETE BEFORE SUBMISSION

If have time, get results for different array contents, and change above to reflect this.

5 Further Investigation

Thread pool

A Code